

Kelva

Gestionnaire de reçus, garanties et abonnements connecté à la boîte mail. Distribué en 5 langues.

Qu'est-ce que c'est

Kelva est une app iOS + Android qui gère numériquement reçus, garanties et abonnements. L'utilisateur connecte plusieurs adresses mail (lecture seule), le courrier entrant arrive dans Kelva, est parsé par l'IA ou scan manuel rapide, classé (facture, garantie, abonnement, contrat), avec rappels. La vue d'ensemble contient tous les reçus, garanties et abos en un seul endroit, plus un coffre optionnel pour documents importants. Avec le partage familial, une famille ouvre une boîte partagée - tout le monde voit ce qui se passe, les priorités se divisent, rien ne tombe à travers les mailles.

À qui ça s'adresse

Adultes 25-55 dans le monde gérant inventaire d'appareils, garanties et abonnements récurrents pour eux ou leur foyer. Audience consommateurs avertis, légèrement DIY, foyers familiaux/de couple. Archétypes de créateurs qui convertissent bien : personal-finance / épargne, IGer organisation familiale, lifehack TikTok dans chaque langue, expliqueurs de droit consommateur (DE avocat-TikTok §437/§438, ailleurs protection consommateur générale), petits recenseurs tech, parent/family-mom influenceurs.

Fonctionnalités clés

- Parsing boîte mail : connecter plusieurs mails en lecture seule, IA auto-classifie ; ou scan photo manuel.
- Vue reçus + garanties + abos filtrable par catégorie, appareil, fournisseur.
- Rappels push : 30 / 7 / 1 jour avant expiration.
- Auto-calc garantie §437 / §438 BGB (bonus DE) ; lookup fabricant ailleurs.
- Partage familial : boîte partagée, priorités par membre.
- Coffre documents + Snap-to-Done.

Modèle économique

- Une seule source de revenu : abonnement Premium annuel.
- Tier gratuit généreux : saisie manuelle, scan photo, rappels, \$438 auto-calc (DE), partage familial.
- Premium déverrouille : classification IA email-inbound (off/warranty-only/all) + fonctionnalités famille avancées.
- Premium gratuit à vie pour affiliés onboarded.
- Langues live : DE / EN / ES / FR / IT.

Prix

- 29,99 USD / an, pas de plan mensuel actuellement.
- Pas d'essai gratuit - le tier gratuit généreux le remplace.
- ASC live depuis mai 2026 ; part affilié toujours 50 % du prix du moment.

Lacune du marché - ce qui manquait

Reçus + garanties + abonnements sont trois problèmes séparés que la plupart des apps traitent isolément. Sub-trackers (Bobby, Subby, Truebill) gèrent les abonnements, ignorent les garanties. Apps reçus (Smart Receipts) scannent le papier mais ne lisent pas la boîte. Trackers garantie (Centriq, Warranty Manager) semblent abandonnés et ne couvrent que les appareils enregistrés. La vraie douleur c'est une personne jonglant entre les trois dans un Gmail débordant, une boîte à chaussures de reçus, et un abo qu'elle ne remarque qu'au prélèvement. Le parsing de la boîte mail comme mécanisme d'input est le déblocage. Le partage familial par-dessus fait de Kelva spécifiquement un produit famille.

Pourquoi maintenant

Deux changements rendent Kelva possible en 2026. D'abord, les coûts Vision et LLM ont assez baissé pour que classer chaque mail entrant par utilisateur soit économiquement viable au prix de 30 USD/an - il y a trois ans ça aurait coûté plus que l'abonnement. Ensuite, les providers mail UE ont stabilisé les flux OAuth IMAP read-only pour accès tiers, donc on peut connecter plusieurs boîtes sans handover de mot de passe risqué.

Rémunération des affiliés

- 50% du revenu net des abonnements Premium pendant 24 mois à partir du premier mois payé de chaque utilisateur attribué.
- Retenue de 30 jours pour les remboursements avant que le paiement devienne réclamable (fenêtre de remboursement Apple / Google).
- Paiement minimum de 50 EUR, mensuel, via Wise.
- Premium gratuit à vie pour les affiliés onboarded (aucune obligation de publication).
- Tableau de bord par handle avec split en direct des clics / installs / abonnements / gains, sans couche d'agence.

Comment fonctionne le tracking - stack d'attribution DIY

La plupart des stacks affiliés tournent sur Branch.io, AppsFlyer ou un SDK d'attribution payant similaire. Nous avons construit le nôtre. Ces outils facturent par install attribué, ils ajoutent une tierce partie à l'expérience de premier lancement de l'utilisateur, et ils rendent les paiements aux influenceurs opaques. Nous voulions un contrôle total de chaque étape, du clic au virement Wise. Le chemin complet a six étapes - clic sur le lien, install, attribution au premier lancement, achat Premium, retenue, paiement. Voici ce qui tourne à chaque étape.

Pourquoi on a construit le nôtre au lieu de Branch.io

Trois raisons. (1) Coût : Branch.io et AppsFlyer facturent 0,20-0,40 USD par install attribué à notre niveau de volume, ce qui aurait absorbé environ 8-12 % de la cagnotte de paiements affiliés. Avec DIY on garde ça pour les affiliés. (2) Vie privée : chaque SDK supplémentaire lit le contexte de premier lancement - c'est un vendor de plus avec des données utilisateur first-touch qu'on ne voulait pas ajouter. Le framework App Tracking Transparency d'Apple a aussi rendu le fingerprinting probabiliste peu fiable depuis iOS 14.5 ; les SDK payants contournent ça avec des heuristiques opaques qu'on ne pouvait pas auditer. (3) Transparence : quand un chiffre du tableau de bord a l'air faux, on lit notre propre code en 10 minutes au lieu de déboguer une black-box tierce. Le trade-off c'est qu'on possède les bugs. Chaque ligne du code d'attribution est dans notre repo.

Android : Google Play install-referrer

Android nous donne le chemin le plus propre. Quand un utilisateur clique sur ton lien, l'install Google Play Store transporte la chaîne referrer originale (un paramètre query URL-encodé depuis la chaîne de redirection) jusqu'au premier lancement via Play Install Referrer API. Au premier lancement l'app lit ``getInstallReferrer().getInstallReferrer()``, on parse handle et source depuis là, et on écrit une ligne d'attribution dans notre table Supabase avec l'événement install. Déterministe. Pas de fingerprinting. Pas de presse-papiers.

iOS warm (app déjà installée)

Si l'utilisateur a déjà Kelva installé, iOS route le tap via un Universal Link directement dans l'app avec le handle dans le chemin URL. On enregistre ``kelva.space/i/*`` comme Universal Link dans le fichier d'entitlements iOS, le CDN d'Apple sert le JSON ``apple-app-site-association`` qu'on publie à la racine du domaine, l'OS route le tap. Dans l'app, le handler de deep-link lit le chemin URL, extrait le handle, écrit la ligne d'attribution. Déterministe.

iOS cold (~99 % du trafic créateur)

Le cas dur - et ~99 % du trafic créateur - c'est iOS cold install. iOS ne donne à l'app fraîchement installée aucun contexte sur comment elle a été atteinte. Tout le business de Branch.io est de résoudre ça avec des heuristiques de device-fingerprinting devenues de plus en plus peu fiables après iOS 14.5. Nous le faisons différemment. Quand un utilisateur clique sur ton lien dans Safari, notre chaîne de redirection touche `kelva.space/i/[handle]`. Cette page écrit un token JSON à courte durée de vie dans le presse-papiers avant de rediriger vers l'App Store. Le token contient : handle, timestamp (issued-at), app-key, version, et une signature HMAC-SHA256 calculée contre un secret côté serveur. Après install, au premier lancement, l'app lit le presse-papiers une fois (Apple montre l'UI système standard de paste-clipboard), parse le JSON, vérifie la signature, accepte le token s'il est signé frais et a moins de 7 jours, écrit la ligne d'attribution. L'utilisateur ne voit aucun écran supplémentaire quand ça marche.

Confirmation optionnelle au premier lancement

Fallback optionnel. Au premier ouverture, après qu'on a lu ce que le presse-papiers a fourni, l'app montre une seule ligne de confirmation : "Invité par @[handle-suggéré] ? Oui / Non". Le champ handle est pré-rempli avec ce que le presse-papiers a suggéré ; si le presse-papiers était vide, le champ est vide et l'utilisateur peut taper. La plupart des utilisateurs clique Oui. Ça attrape le cas limite où la dialogue d'accès au presse-papiers d'Apple a été refusé, l'utilisateur a copié autre chose entre le tap et l'install, ou le presse-papiers a expiré. L'étape est activable par app.

Webhook RevenueCat -> queue de paiement

Les achats Premium passent par RevenueCat. RC gère les reçus Apple StoreKit et Google Play Billing, le dedup, la détection de remboursement, et convertit tout en un format d'événement webhook cohérent. Quand un événement ``INITIAL_PURCHASE``, ``RENEWAL``, ``PRODUCT_CHANGE`` ou ``CANCELLATION`` tire, RC poste JSON à notre Edge function Supabase ``affiliate-rc-webhook``. La fonction (1) vérifie l'en-tête de signature RC contre notre secret partagé, (2) idempotency-check sur l'ID événement contre ``processed_webhook_events``, (3) cherche la ligne d'attribution de l'utilisateur, (4) calcule 50 % du revenu net après le cut Apple/Google de 15-30 %, (5) crédite la table ``affiliate_earnings`` avec handle, app, gross, cut, net, our-share, date holdback-until, et l'ID événement source. Le tout dans une transaction Postgres.

Pourquoi ça marche comme pitch affilié

Garanties expirées et abos oubliés coûtent du vrai argent dans chaque marché - pas qu'un problème DACH. La démo du parsing mail est visuellement forte dans chaque langue. L'auto-calc §437 / §438 BGB est un bonus DE-spécifique que les avocats-TikToker démontrent concrètement ; audiences non-DE ont l'angle protection consommateur générique. 29,99 USD annuels convertissent au-dessus de la moyenne parce que l'alternative (une garantie ratée = 200-500 EUR perdus) est concrète et marquante.

Pour les posts avec lien affilié : #ad / #sponsorisé / Paid Partnership (FTC, droit européen).